

Low-Level Design (LLD): RAG Customer Support Assistant

LangGraph · ChromaDB · HITL · OpenAI

Field	Value
Document Type	Low-Level Design (LLD)
Module Coverage	All 8 modules (ingestion → HITL)
Implementation Language	Python 3.10+
Status	Final

1. Module-Level Design

Document Processing Module

src/ingestion.py

- load_pdf(path) → List[Document]
- Calls PyPDFLoader, returns one Document per page with metadata.

Chunking Module

src/ingestion.py

- chunk_documents(docs) → List[Document]
- RecursiveCharacterTextSplitter(size=500, overlap=100). Splits on
 - >
 - > . > space.

Embedding Module

src/ingestion.py

- get_embeddings() → HuggingFaceEmbeddings
- Model: all-MiniLM-L6-v2, 384-dim, cosine-normalized, CPU inference.

Vector Storage Module

src/ingestion.py

- embed_and_store(chunks, reset) → Chroma
- Persists to ./chroma_db. Collection: support_knowledge_base.

Retrieval Module

src/retrieval.py

- retrieve(query, top_k=4) → List[(Document, float)]
- similarity_search_with_relevance_scores. Threshold: 0.6.

Query Processing Module

src/llm.py

- generate_answer(question, context) → str
- Builds SystemMessage + HumanMessage with prompt template. Calls LLM.

Graph Execution Module

src/workflow.py

- `build_graph()` → `CompiledGraph`
- 6-node `StateGraph`. `run_query(q)` → `GraphState` is the public API.

HITL Module

`src/hitl.py`

- `create_ticket(query, reason)` → `EscalationTicket`
- `resolve_ticket(id, response).should_escalate(q, conf, ans)` → `(bool, reason)`.

2. Data Structures

2.1 Document Representation

```
class Document:
    page_content: str    # Raw text of the chunk
    metadata: dict       # {source: str, page: int}
```

2.2 Chunk Format

```
# After RecursiveCharacterTextSplitter
chunk = Document(
    page_content = 'Return policy allows 30-day refund...',
    metadata = {'source': 'manual.pdf', 'page': 3}
)
```

2.3 Embedding Structure

```
# ChromaDB internal storage
{
    'id' : 'uuid-xxxx',
    'embedding' : [0.021, -0.134, ..., 0.087], # 384 floats
    'document' : 'chunk text',
    'metadata' : {'source': 'file.pdf', 'page': 2}
}
```

2.4 Query-Response Schema

```
{
    'query'          : str,                # original user question
    'retrieved'      : [(Document, float), ...],
    'context'        : str,                # assembled context for LLM
    'llm_answer'     : str,                # raw LLM response
    'escalated'      : bool,
    'ticket_id'      : Optional[int],
    'final_answer'   : str                 # delivered to user
}
```

2.5 GraphState (LangGraph State Object)

```
class GraphState(TypedDict):
    user_query      : str
```

```

retrieved_chunks : Optional[List[Tuple[Document, float]]]
context_string   : Optional[str]
low_confidence   : Optional[bool]
llm_answer       : Optional[str]
escalate         : Optional[bool]
escalation_reason : Optional[str]
final_answer     : Optional[str]
ticket_id        : Optional[int]

```

2.6 EscalationTicket Dataclass

```

@dataclass
class EscalationTicket:
    ticket_id      : int
    user_query     : str
    escalation_reason : str
    timestamp      : float # time.time()
    status         : str   # 'OPEN' | 'RESOLVED'
    human_response : Optional[str]

```

3. Workflow Design (LangGraph)

3.1 Node Definitions

Node	Input Fields Read	Output Fields Written	Side Effects
input_node	user_query	user_query (cleaned)	None
retrieve_node	user_query	retrieved_chunks, context_string, low_confidence	ChromaDB query
generate_node	user_query, context_string	llm_answer	LLM API call
route_node	user_query, low_confidence, llm_answer	escalate, escalation_reason	None
answer_node	llm_answer	final_answer	None
escalate_node	user_query, escalation_reason	final_answer, ticket_id	Creates HITL ticket

3.2 Edge Definitions

From	To	Edge Type	Condition
START	input_node	Direct	Always
input_node	retrieve_node	Direct	Always
retrieve_node	generate_node	Direct	Always
generate_node	route_node	Direct	Always
route_node	answer_node	Conditional	escalate == False

route_node	escalate_node	Conditional	escalate == True
answer_node	END	Direct	Always
escalate_node	END	Direct	Always

3.3 State Transition Diagram

START → input → retrieve → generate → route → [answer|escalate] → END

4. Conditional Routing Logic

4.1 should_escalate() Decision Tree

- 1. IF any escalation keyword in query.lower(): → ESCALATE (reason: 'Sensitive keyword: ')
- 2. ELIF low_confidence == True: → ESCALATE (reason: 'Low retrieval confidence')
- 3. ELIF 'don't have enough information' in llm_answer.lower(): → ESCALATE (reason: 'LLM expressed insufficient information')
- 4. ELIF len(query.split()) > 30: → ESCALATE (reason: 'Complex multi-part query')
- 5. ELSE: → ANSWER DIRECTLY

4.2 Escalation Keywords List

refund · lawsuit · legal · manager · urgent · broken · dangerous · fraud · complaint · billing error

4.3 Confidence Threshold

RELEVANCE_THRESHOLD = 0.6 (cosine similarity). If the best-matching chunk scores below this, the knowledge base is deemed insufficient and escalation is triggered.

5. HITL Design

5.1 Escalation Trigger: Escalation is triggered inside route_node when should_escalate() returns True. The graph then routes to escalate_node instead of answer_node.

5.2 Ticket Lifecycle

Stage	Action	Result
Trigger	route_node sets escalate=True	Redirects to escalate_node
Create	create_ticket(query, reason) called	EscalationTicket in _ticket_store
Queue	Ticket added to _pending_queue	Human agent notified (production)
Resolve	resolve_ticket(id, response) called	Status → RESOLVED, response stored
Integrate	Human response returned as final_answer	Displayed to user with ticket ID

5.3 Production Integration: In production, replace `simulate_human_response()` with a webhook callback from a real ticketing system (Zendesk, Jira Service Desk, Freshdesk). The `resolve_ticket()` function serves as the integration point for the callback handler.

6. API / Interface Design

6.1 Public API: `run_query()`

```
# src/workflow.py
def run_query(user_query: str) -> GraphState:
```

6.2 CLI Interface

Command	Arguments	Description
<code>python main.py demo</code>	none	Run scripted demo with synthetic KB
<code>python main.py ingest</code>	<code><pdf_path> [--reset]</code>	Ingest a PDF into ChromaDB
<code>python main.py chat</code>	none	Interactive REPL chat session
<code>python main.py query</code>	"Your question"	Single query, print answer, exit

6.3 Input / Output Format

```
# Input
user_query: str # plain text, e.g. 'What is the return policy?'

# Output (normal answer)
final_answer: str # 'Our return policy allows...'

# Output (escalated)
final_answer: str # '■ Your query has been escalated...'
# ' Ticket ID : #1001'
# '■ Human Agent Response: ...'
```

7. Error Handling

Error Scenario	Where Handled	Handling Strategy
PDF file not found	<code>ingestion.py / ingest_pdf()</code>	Raises <code>FileNotFoundError</code> with descriptive message.
No relevant chunks found	<code>retrieval.py / is_low_confidence()</code>	Returns <code>low_confidence=True</code> . <code>route_node</code> escalates.
ChromaDB empty	<code>retrieval.py / retrieve()</code>	Returns empty list → escalation triggered.
LLM API failure / timeout	<code>llm.py / generate_answer()</code>	<code>try/except</code> catches exceptions. Returns fallback.
Missing <code>OPENAI_API_KEY</code>	<code>llm.py / _get_llm()</code>	Automatically falls back to <code>MockLLM</code> .

Invalid / empty user query	workflow.py / input_node()	Strips whitespace. Substitutes '(empty query)'.
HITL ticket ID not found	hitl.py / resolve_ticket()	Logs warning and returns None.